

Diseño y Análisis de Algoritmos

Set de Problemas 2

Profesor: Felipe Ruiz

Ayudante: Cristian Nettle

Multiplicación de matrices — Método de Strassen

Considere el producto de dos matrices cuadradas $n \times n$ usando *Strassen*: se realizan 7 multiplicaciones recursivas de submatrices de tamaño $n/2$ y la combinación cuesta cn^2 por alguna constante $c > 0$ (sumas/restas de submatrices). Suponga que $n = 2^k$ para algún $k \in \mathbb{N}$ y que $T(n) = \Theta(1)$ para $n \leq n_0$.

- (a) Modele la ecuación de recurrencia $T(n)$ para el tiempo total.
- (b) Resuelva **por expansión** la recurrencia del apartado (a): muestre sustituciones sucesivas, identifique el patrón, elija t adecuado, evalúe los términos resultantes y concluya la complejidad temporal en notación $\Theta(\cdot)$.

Exponenciación binaria

Para calcular a^n (con n entero no negativo), la *exponenciación binaria* divide el exponente por 2: resuelve un subproblema de tamaño $n/2$ y combina en tiempo constante $c > 0$ (multiplicaciones constantes; si n es impar, un factor adicional de a que no altera la cota). Suponga que $n = 2^k$ para algún $k \in \mathbb{N}$ y que $T(1) = \Theta(1)$.

- (a) Modele la ecuación de recurrencia $T(n)$ para el tiempo total en función del exponente n .
- (b) Resuelva **por expansión** la recurrencia del apartado (a): muestre sustituciones sucesivas, identifique el patrón, elija t adecuado y concluya la complejidad temporal en notación $\Theta(\cdot)$.

Máximo subarreglo

Dado un arreglo $A[1..n]$ (con números positivos y/o negativos), determine el **subarreglo contiguo** de suma máxima y su valor. *Convención: el subarreglo es no vacío. En este ejercicio pide un enfoque de dividir y conquistar; soluciones lineales (p. ej., Kadane) no sustituyen lo pedido en (a) y (b).*

- (a) Describa en palabras un algoritmo de *dividir y conquistar*: indique cómo particiona el problema, qué información mínima devuelve cada subproblema para poder *combinar* en tiempo lineal, cuál es el caso base y cómo garantiza que la combinación es $O(n)$.
- (b) Modele la recurrencia $T(n)$ y concluya la cota asintótica del tiempo de ejecución, justificando su resultado.
- (c) Ilustre la ejecución sobre el arreglo $A = \langle -2, 1, -3, 4, -1, 2, 1, -5, 4 \rangle$, mostrando la entrada y salida de los subproblemas relevantes y el resultado final.

Conteo de inversiones

Dados $A[1..n]$, una *inversión* es un par (i, j) tal que $1 \leq i < j \leq n$ y $A[i] > A[j]$. Calcule el **número total** de inversiones en A .

- Describa en palabras un algoritmo de *dividir y conquistar*: explique cómo particiona el arreglo, qué información devuelve cada subproblema y cómo realiza la *combinación* en tiempo lineal para acumular el conteo total de inversiones.
- Modele la recurrencia $T(n)$ y establezca la complejidad asintótica del algoritmo, justificando su resultado.
- Ilustre la ejecución sobre el arreglo $A = \langle 2, 4, 1, 3, 5 \rangle$, mostrando cómo se acumula el conteo durante la fase de combinación y el total final.

Bowling

Se tiene una línea de n pinos con puntajes (posibles negativos) $B[1..n]$. En la posición i se puede: (1) derribar *solo* el pino i y sumar $B[i]$; (2) derribar el *par consecutivo* $(i-1, i)$ en una misma jugada y sumar $B[i-1] \cdot B[i]$; o (3) *omitir* i . Las jugadas no pueden superponerse: *cada pino puede participar a lo más en una jugada* (si se toma $(i-1, i)$, no se pueden tomar $i-1$ ni i por separado). Determine la **puntuación máxima** posible.

- Para el ejemplo

$$B[1..6] = \langle 4, -3, 7, -2, 5, -6 \rangle,$$

calcule **a mano** el valor óptimo de cada prefijo $F(i)$ (máximo usando solo los primeros i pinos) para $i = 1, 2, \dots, 6$, e indique el valor final $F(6)$. Complete además los **casos base**: $F(0) = \underline{\hspace{1cm}}$, $F(1) = \underline{\hspace{1cm}}$.

i	1	2	3	4	5	6
$F(i)$						

Nota: Como $B[i]$ puede ser negativo, omitir puede ser óptimo en algunos pasos.

- Complete los espacios en blanco de la definición de $F(i)$, donde $F(i)$ representa la puntuación máxima usando solo los primeros i pinos con las decisiones permitidas (tomar i solo, tomar el par $(i-1, i)$ u omitir i):

$$F(i) = \begin{cases} \underline{\hspace{1cm}}, & \text{si } i = 0, \\ \underline{\hspace{1cm}}, & \text{si } i = 1, \\ \max \{ \underline{\hspace{1cm}}, \underline{\hspace{1cm}}, \underline{\hspace{1cm}} \}, & \text{si } 2 \leq i \leq n, \end{cases} \quad i \in \{0, 1, \dots, n\}.$$

- Usando pseudocódigo, implemente $\text{MAXBOWLING}(B[1..n], n)$ que calcule $F(n)$ en $O(n)$ tiempo y $O(n)$ espacio adicional. Puede mencionar cómo optimizar el espacio a $O(1)$ manteniendo solo $F(i-1)$ y $F(i-2)$.

Corte de barra

Se tiene una barra de longitud n . El precio de una pieza de longitud i está dado por $P[i] \geq 0$. Se puede cortar la barra en piezas de longitudes enteras cuya suma sea la longitud total (o bien no cortar). No hay costo de corte. Dado $P[1..n]$, determine el **ingreso máximo** posible.

- (a) Para el ejemplo

$$P[1..8] = \langle 1, 5, 8, 9, 10, 17, 17, 20 \rangle,$$

calcule **a mano** el valor óptimo $F(i)$ (ingreso máximo para barra de longitud i) para $i = 1, 2, \dots, 8$ e indique $F(8)$. Complete además los **casos base**: $F(0) = \underline{\hspace{1cm}}$, $F(1) = \underline{\hspace{1cm}}$.

i	1	2	3	4	5	6	7	8
$F(i)$								

Nota: $P[i]$ no tiene por qué ser creciente; cortar (o no) puede ser óptimo según la tabla.

- (b) Complete los espacios en blanco de la definición de $F(i)$, donde $F(i)$ representa el ingreso máximo para longitud i :

$$F(i) = \begin{cases} \underline{\hspace{1cm}}, & \text{si } i = 0, \\ \underline{\hspace{1cm}}, & \text{si } i = 1, \\ \max_{1 \leq k \leq i} \{ \underline{\hspace{1cm}} \}, & \text{si } 2 \leq i \leq n, \end{cases} \quad i \in \{0, 1, \dots, n\}.$$

- (c) Usando pseudocódigo, implemente $\text{CUTROD}(P[1..n], n)$ que calcule $F(n)$ en $O(n^2)$ tiempo y $O(n)$ espacio adicional (puede comentar reconstrucción de cortes).